



UNIVERSIDAD DEL VALLE
FACULTAD DE INGENIERÍA

Desarrollo del Primer Examen Parcial de Sistemas Operativos

SISTEMAS OPERATIVOS

Docente: Jefferson Amado Peña Torres

David Taborda Montenegro
202242264 - 3743
Ingeniería de Sistemas

Santiago de Cali

Junio de 2026

1. Introducción.

Para el desarrollo de esta actividad evaluativa, cuyo enfoque ha sido más inclinado hacia el ámbito práctico, se ha planteado la necesidad de analizar y desarrollar uno de los pilares fundamentales en la arquitectura de los sistemas operativos modernos: la gestión y planificación de procesos. En entornos donde se evidencian concurrencias, la asignación eficiente del recurso de cómputo de la CPU es una tarea crítica que impacta directamente en la percepción de interactividad del usuario y en el rendimiento global del hardware y el software, por lo que la propuesta brindada en este trabajo, se fundamenta en la implementación rigurosa y el diseño de un simulador para el algoritmo de **Planificación de Colas Multinivel (MLQ)**, por sus siglas en inglés), utilizando para ello el paradigma de Programación Orientado a Objetos como mecanismo conductor para encapsular el comportamiento del código de manera segura, escalable y efectiva al interactuar con los procesos a administrar.

Particularmente, se enfatiza el potencial de este algoritmo de planificación para segregar los procesos del sistema en función de sus necesidades operativas, permitiendo que convivan de forma armoniosa políticas expropiativas, basadas en divisiones de tiempo (quántums), con esquemas no-expropiativos, basados en prioridades. Para esta labor, la solución aquí diseñada aborda conceptualmente un esquema de tres niveles jerárquicos independientes, cuyo comportamiento se fundamenta en un manejo práctico de un reloj global unificado y un sistema de sincronización dada la concurrencia. Ésto posibilita expresar comportamientos de asignación de forma directa, considerando el “salto” de tiempos donde no se están ejecutando algunas de estas instancias activas, garantizando la correctitud analítica y matemática del sistema, reflejada en el cálculo de las métricas correspondientes.

En ese orden de ideas, es relevante mencionar el comportamiento y los parámetros que gobernarán la simulación del algoritmo MLQ para este informe, los cuales consisten en tres colas estructuradas bajo el siguiente

esquema:

- **Cola 1 (Q1):** Gobernada por el algoritmo *Round Robin* con un cuántum de tres unidades de tiempo (RR(q=3)).
- **Cola 2 (Q2):** Configurada también con el algoritmo *Round Robin* con un cuántum de cinco unidades de tiempo (RR(q=5)).
- **Cola 3 (Q3):** Destinada a utilizar el algoritmo de *Prioridad No Expropiativa*, con una jerarquía de (5 > 1), donde el valor 5 es el más prioritario y 1 es el menos prioritario.

Así, una vez se comprende esta distribución arquitectónica, se da paso a las argumentaciones y desgloses detallados de la implementación práctica.

2. Implementación del Simulador.

Conociendo que el simulador consiste en el algoritmo de planificación MLQ con tres colas, su diseño no se concibió como un script secuencial, por el contrario, se aplicó una programación orientada a objetos robusta dentro del lenguaje de programación Python, fragmentando el dominio del problema en tres estructuras de clases interconectadas, modelando fielmente las entidades que intervienen en un sistema operativo en estos casos:

⇒ **Clase 'Proceso':**

Su principal función es la de modelar el Bloque de Control de Procesos (PCB) que dispondrá cada entidad despachable involucrada, comportándose como el objeto que contiene la información detallada de una tarea en ejecución. En ese orden, esta clase actúa abstrayendo y protegiendo, mediante sus atributos, los datos nativos ingresados desde los archivos planos de entrada, como la Etiqueta, el tiempo de cómputo requerido (BT), el instante en que llega a la cola de listos (AT), la cola a la que irá (Q) y la prioridad (Px) que tendrá cada proceso.

Se optó por este diseño, debido a que permite manejar internamente el estado de sus propias métricas (WT, CT, RT, TAT) al incorporar métodos atómicos como *'registrar_primer_toque()'* y *'finalizar()'*, pues el objeto será capaz de capturar con precisión el momento exacto en el que “toca” la CPU por primera vez y consolidar matemáticamente sus tiempos netos en el sistema en el instante preciso de su finalización, asemejándose a la manera como se abordaron las obtenciones de estos valores a lo largo del curso.

⇒ Clase 'ColaMLQ':

Esta clase representa una de las estructuras de datos principales dentro del código, pues se encarga de modelar el comportamiento de la cola donde se ordena y determina qué proceso ingresará a la CPU. Su importancia y característica esencial, radica en que, en vez de manipular listas “normales” en el flujo principal, ColaMLQ encapsula un **arreglo dinámico** interno, cuyo fin es restringir el comportamiento de los arreglos a usar, para que operen de manera pura como una **cola FIFO** (First-In, First-Out), es decir, que los procesos que están en el estado 'listo' sean atendidos por el procesador en el orden en que llegan, respetando claro el algoritmo interno de cada cola. Esto se comprueba durante el ingreso de procesos a través del método '*insertar()*', donde se ejecuta una operación de Enqueue (*.append()*) al final del arreglo (el último que llega será el último en atenderse, dentro de su respectiva cola); mientras que, al momento de despacho, se extraen los procesos a ejecutar mediante una operación Dequeue (*.pop(0)*), comenzando por la cabeza del arreglo.

Además del comportamiento estructural de tipo FIFO, la clase evidencia en su método principal '*ejecutar_proceso()*' un comportamiento adaptable al algoritmo interno de cada cola, el cual muta de manera transparente según la política configurada en su inicialización: si la cola está parametrizada como Round Robin, el método calcula la fracción de tiempo a ejecutar (restando el cuántum del tiempo restante del proceso). Si la ráfaga no es suficiente para liquidar el proceso, la estructura lo reencola de forma circular al final de la cola FIFO, formando así la estructura implícita de **cola circular** que caracteriza a este algoritmo expropiativo. Por su parte, si la cola opera bajo Prioridad No Expropiativa, el método ejecuta una ordenación interna inmediata basada en el atributo de prioridad antes de extraer el elemento, a través de una función anónima (lambda), transformando momentáneamente la estructura en una **cola de prioridad**, garantizando el cumplimiento de las reglas vistas en el curso sin alterar el flujo de ejecución principal.

⇒ Clase 'SimuladorMLQFinal':

Finalmente, se dispone de la clase principal para el simulador, cuya implementación es la más interesante y ciertamente compleja de estructurar. Como se trata del algoritmo MLQ, su funcionamiento radica en mantener la sincronía temporal entre las ejecuciones internas de las colas. Entonces, para solucionar esto, la clase actúa como el director de orquesta que controla un reloj global unificado, basado en un ciclo durante la ejecución de los procesos.

De esta manera, en busca de impedir expropiaciones arbitrarias en mitad de un cuántum en las colas Q1 y Q2 o la interrupción de la ejecución de los procesos por su prioridad en la cola Q3, se diseñó un mecanismo de sincronización mediante métodos que permiten que la ejecución de cada cola sea continua, usando el concepto de *funciones callback*. Ahondando un poco, este enfoque permite que cada cola mantenga una ejecución continua sin perder el rastro del sistema general. Específicamente, al delegar el control de la CPU a una cola, el simulador le pasa como argumento una referencia a su método privado '*_registrar_arribos()*'. Así, a medida que el reloj de la CPU avanza dentro de esa cola, ésta invoca el callback de forma asíncrona para escanear y encolar inmediatamente los nuevos procesos en sus respectivos niveles. Al finalizar la ráfaga actual, el bucle principal detecta una ruptura crítica (*break*) y reevalúa la jerarquía desde el nivel más alto. Esto garantiza el cumplimiento estricto del esquema de "oleadas" jerárquicas del curso: se vacía por completo Q1, luego Q2 y finalmente Q3, tal como se ha manejado en el curso, preservando cada valor calculado e ingresado en nuevos archivos de texto.

Con todo lo anterior, es comprobable que el código, mediante una POO, permite desarrollar la simulación de forma real y medible el algoritmo MLQ y obtener las métricas respectivas de cada procedimiento temporal, basado en la manera enseñada presencialmente.

3. Pruebas para Validación del Funcionamiento.

Ahora bien, con el fin de validar exhaustivamente la correctitud de los resultados arrojados por el simulador frente al modelo teórico desarrollado “a mano” para obtener las métricas, la implementación fue sometida a pruebas con cuatro archivos de entrada diferenciados: tres correspondientes a los alojados en la carpeta de Google Drive compartida en el enunciado, específicamente los archivos ‘mlq001.txt’, ‘mlq006.txt’ y ‘mlq019.txt’, así como un archivo de texto personalizado con un escenario original, manteniendo la estructura de los archivos antes mencionados.

Para esta sección, se desarrolló todo el proceso de cálculo de métricas manualmente (a través de un diagrama de Gantt y el registro de valores en tablas para cada archivo, aplicando claramente el algoritmo mencionado desde el inicio), para comparar si efectivamente el simulador ejecuta bien su procedimiento y permita obtener exactamente las mismas métricas esperadas. A continuación, se comparten la tabla y el diagrama correspondiente a cada archivo junto con una captura de pantalla del contenido dentro del archivo de texto resultante, buscando observar que los datos coinciden exactamente entre ambos métodos. Cabe aclarar que los datos contenidos dentro de cada archivo de texto, se visualizan dentro de cada una de las tablas, específicamente dentro de las columnas “Etiqueta”, “BT”, “AT”, “Q” y “Px”.

- Archivo 'mlq001.txt'.

De manera teórica:

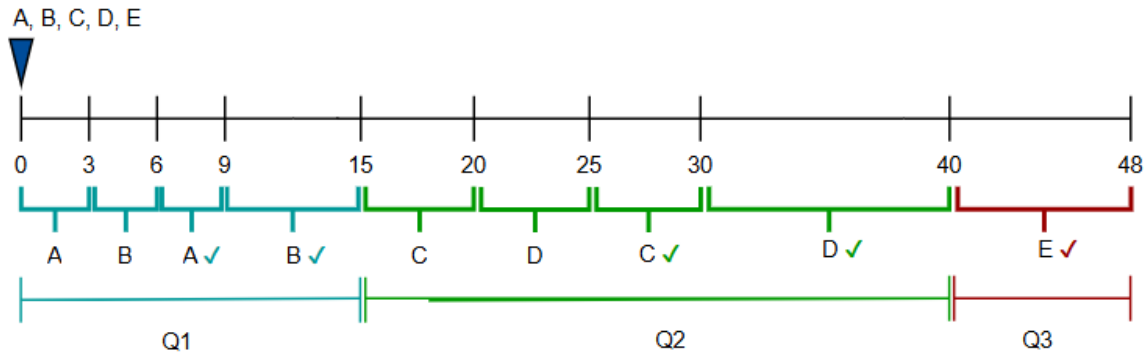


Imagen 1. Diagrama de Gantt para el archivo *mlq001.txt*

Etiqueta	BT	AT	Q	Px	WT	CT	RT	TAT
A	6	0	1	5	3	9	0	9
B	9	0	1	4	6	15	3	15
C	10	0	2	3	20	30	15	30
D	15	0	2	3	25	40	20	40
E	8	0	3	2	40	48	40	48
				$\bar{X}$	18,8	28,4	15,6	28,4

Tabla 1. Tabla de métricas para el archivo *mlq001.txt*

Arrojado por el sistema:

```
# Archivo: mlq001.txt
# Etiqueta; BT; AT; Q; Px; WT; CT; RT; TAT
A;6;0;1;5;3;9;0;9
B;9;0;1;4;6;15;3;15
C;10;0;2;3;20;30;15;30
D;15;0;2;3;25;40;20;40
E;8;0;3;2;40;48;40;48
# WT=18.8; CT=28.4; RT=15.6; TAT=28.4;

Process finished with exit code 0
```

Imagen 2. Salida del simulador para el archivo *mlq001.txt*

- Archivo 'mlq006.txt'.

De manera teórica:

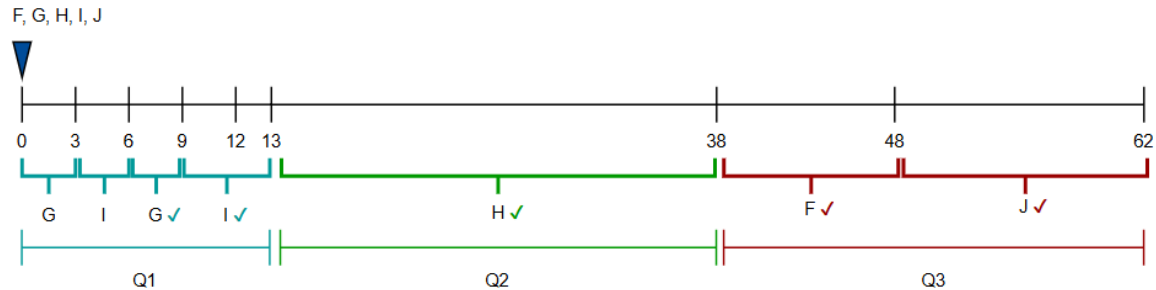


Imagen 3. Diagrama de Gantt para el archivo *mlq006.txt*

Etiqueta	BT	AT	Q	Px	WT	CT	RT	TAT
F	10	0	3	5	38	48	38	48
G	6	0	1	4	3	9	0	9
H	25	0	2	3	13	38	13	38
I	7	0	1	2	6	13	3	13
J	14	0	3	1	48	62	48	62
				$\bar{x}$	21,6	34	20,4	34

Tabla 2. Tabla de métricas para el archivo *mlq006.txt*

Arrojado por el sistema:

```
# Archivo: mlq006.txt
# Etiqueta; BT; AT; Q; Pr; WT; CT; RT; TAT
F;10;0;3;5;38;48;38;48
G;6;0;1;4;3;9;0;9
H;25;0;2;3;13;38;13;38
I;7;0;1;2;6;13;3;13
J;14;0;3;1;48;62;48;62
# WT=21.6; CT=34.0; RT=20.4; TAT=34.0;

Process finished with exit code 0
```

Imagen 4. Salida del simulador para el archivo *mlq006.txt*

- Archivo 'mlq019.txt'.

De manera teórica:

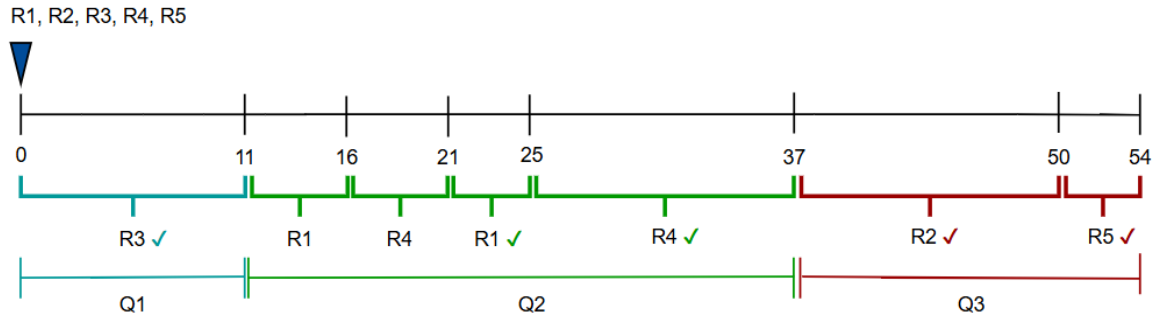


Imagen 5. Diagrama de Gantt para el archivo *mlq019.txt*

Etiqueta	BT	AT	Q	Px	WT	CT	RT	TAT
R1	9	0	2	5	16	25	11	25
R2	13	0	3	4	37	50	37	50
R3	11	0	1	3	0	11	0	11
R4	17	0	2	2	20	37	16	37
R5	4	0	3	1	50	54	50	54
				$\bar{x}$	24,6	35,4	22,8	35,4

Tabla 3. Tabla de métricas para el archivo *mlq019.txt*

Arrojado por el sistema:

```
# Archivo: mlq019.txt
# Etiqueta; BT; AT; Q; Px; WT; CT; RT; TAT
R1;9;0;2;5;16;25;11;25
R2;13;0;3;4;37;50;37;50
R3;11;0;1;3;0;11;0;11
R4;17;0;2;2;20;37;16;37
R5;4;0;3;1;50;54;50;54
# WT=24.6; CT=35.4; RT=22.8; TAT=35.4;

Process finished with exit code 0
```

Imagen 6. Salida del simulador para el archivo *mlq019.txt*

- Archivo 'mlq_personalizado.txt'.

De manera teórica:

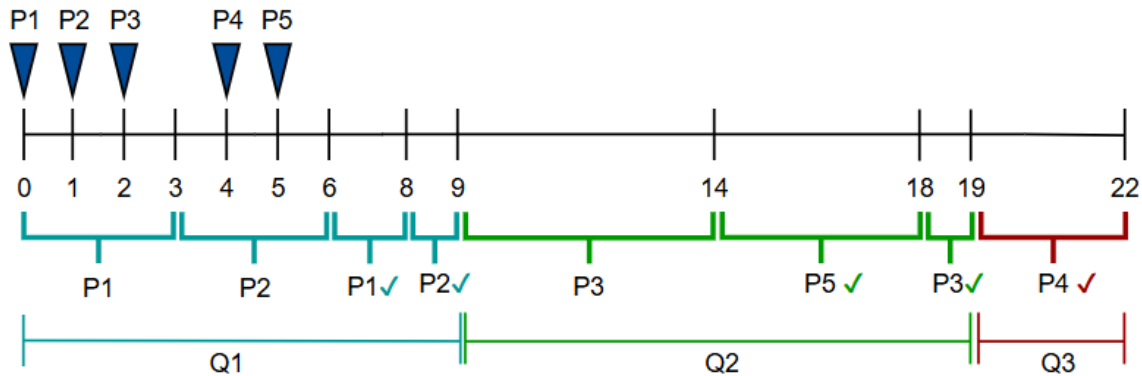


Imagen 7. Diagrama de Gantt para el archivo *mlq_personalizado.txt*

Etiqueta	BT	AT	Q	Px	WT	CT	RT	TAT
P1	5	0	1	3	3	8	0	8
P2	4	1	1	2	4	9	3	8
P3	6	2	2	4	11	19	9	17
P4	3	4	3	1	15	22	19	18
P5	4	5	2	5	9	18	14	13
				$\bar{X}$	8,4	15,2	9	12,8

Tabla 4. Tabla de métricas para el archivo *mlq_personalizado.txt*

Arrojado por el sistema:

```
# Archivo: mlq_personalizado.txt
# Etiqueta; BT; AT; Q; Px; WT; CT; RT; TAT
P1;5;0;1;3;3;8;0;8
P2;4;1;1;2;4;9;3;8
P3;6;2;2;4;11;19;9;17
P4;3;4;3;1;15;22;19;18
P5;4;5;2;5;9;18;14;13
# WT=8.4; CT=15.2; RT=9.0; TAT=12.8;

Process finished with exit code 0
```

Imagen 8. Salida del simulador para el archivo *mlq_personalizado.txt*

Tal como se ha comprobado visual y teóricamente, ambos procedimientos (teórico y algorítmico) arrojan los mismos valores exactos para cada uno de los archivos manejados, demostrando la correcta implementación del simulador aquí abordado.

4. Uso responsable y transparente de la IA.

Dando cumplimiento estricto a los lineamientos de originalidad y transparencia solicitados para esta entrega, y abordando esta sección de manera más personal y honesta, presento los registros del uso que realicé de Inteligencia Artificial, como herramienta de apoyo técnico para la optimización de la calidad del software y la exploración de alternativas de compatibilidad en mi entorno de desarrollo. De esta manera, detallaré los prompts que empleé y la respectiva personalización humana aplicada para cada uno.

Prompt 1: Optimización de Calidad y Documentación Industrial.

Prompt suministrado a la IA: *"Hola Gemini, buenas tardes. Tengo un código funcional en Python que implementa el algoritmo de planificación MLQ para mi curso de Sistemas Operativos. El código ya resuelve la lógica de colas de manera correcta, pero quiero robustecerlo bajo el paradigma de Programación Orientada a Objetos (POO) fuerte. Necesito tu ayuda para inyectarle algunas buenas prácticas de documentación a este código, con el fin de poder entregar un trabajo legible para cualquier persona, en español y que siga estándares de la industria actuales. Además, quiero que las descripciones de los métodos reflejen los conceptos teóricos de la materia, como Bloque de Control de Procesos (PCB), operaciones Enqueue/Dequeue y colas de prioridad".*

Personalización y ajuste propio realizado: Tras la generación de la estructura documental por parte de la IA, realicé una revisión de ingeniería para adaptar los comentarios del lenguaje técnico a los términos vistos en el curso. Específicamente, eliminé terminologías anglosajonas como "vector" o "buffer" en la descripción de las colas, o palabras como "listas" por "arreglos", que es más preciso en la implementación que desarrollé. Así mismo, verifiqué que la nomenclatura de las métricas (WT, CT, RT, TAT) se mantuviera idéntica al estándar del enunciado, dejando estas mejoras evidentes en la documentación del código entregado.

Prompt 2: Compatibilidad del Entorno de Entrada del IDE.

Prompt suministrado a la IA: *"Actualmente mi simulador lee los archivos utilizando parámetros estrictos desde la consola del sistema operativo mediante sys.argv. Sin embargo, esto me genera advertencias y dificultades para ejecutar pruebas rápidas directamente con el botón de reproducción ('Play') en mi IDE IntelliJ IDEA Ultimate, a menos que configure argumentos de ejecución manuales. ¿Qué estructura limpia puedo implementar en el bloque 'if __name__ == '__main__':' para que el script sea híbrido? Es decir, que si le paso un argumento por terminal lo procese, pero si lo ejecuto directamente en el IDE, cargue también el archivo que necesite".*

Personalización y ajuste propio realizado: Implementé la solución de control condicional generada por la IA Gemini, utilizando el módulo sys y una variable llamada 'archivo_origen' para que, localmente, también me funcionara en mi IDE. Así mismo, durante las pruebas en mi entorno local de IntelliJ, identifiqué y corregí una advertencia generada por el IDE sobre el argumento que extrae el código al invocarse por consola, lo que garantizó la correctitud del código al utilizarlo de ambas maneras.

Adjunto la evidencia aquí:

```
if __name__ == "__main__":
    # Configuración de entrada inteligente, para funcionar en llamados tanto por consola interna como externa.
    if len(sys.argv) >= 2:
        archivo_objetivo: str = sys.argv[1]
    else:
        archivo_objetivo = "mlq001.txt" # Por defecto, se carga el archivo de prueba mlq001.txt

    sim = SimuladorMLQFinal()
    sim.cargar_procesos(archivo_objetivo)
    sim.simular()
```

5. Conclusiones.

A partir del desarrollo de este componente práctico, se evidencia con claridad la pertinencia y potencia del uso de herramientas como el reloj global unificado y el mecanismo de sincronización mediante funciones callback, las cuales demostraron ser un medio altamente expresivo y preciso para la emulación, control y secuenciación de procesos estructurados en el algoritmo de colas multinivel (MLQ), evitando por completo el uso de variables globales desincronizadas, bloqueos mutuos o instrucciones de control de tiempo ajenas a la lógica del despacho jerárquico observado en el curso. El bucle principal en la clase `SimuladorMLQFinal`, operó como un motor natural de avance y selección de tareas, favoreciendo construcciones ligadas directamente al dominio de la planificación real, especialmente en el análisis de oleadas continuas y la captura inmediata de arribos en alta prioridad, proporcionando además una exactitud en el cálculo y devolución de las métricas solicitadas, tal como se comprobó en la sección de pruebas, otorgando también una buena legibilidad y fiabilidad sobre la implementación.

En ese mismo orden, las colecciones de carácter estructurado bajo arreglos dinámicos poseen también una capacidad muy potente al momento de agrupar y despachar procesos, puesto que permiten organizarlos de forma secuencial y, debido a las restricciones de su implementación, actúan fielmente bajo la política FIFO de manera pura y adaptable a las necesidades de cada algoritmo interno, como RR o Prioridad No Expropiativa. Con tal característica, se origina un acceso a los datos muy directo y controlado, lo cual reduce la complejidad algorítmica y los recursos de hardware empleados para los procesos que impliquen transiciones de estado en la CPU, lo cual refuerza la recomendación y uso de este tipo de abstracciones encapsuladas para todo software de simulación, en especial bajo las directrices de diseño y las reglas de prioridad tratadas en el curso.

Y, globalmente, la experiencia de implementación refuerza la idea de que el diseño basado en el paradigma Orientado a Objetos no sólo es conceptualmente más limpio, sino también más robusto, seguro y escalable a nivel lógico. La posibilidad de razonar formalmente sobre el comportamiento de las entidades del sistema operativo (como el Bloque de Control de Procesos y las colas circulares y de prioridad de los algoritmos), junto con la orquestación transparente de las políticas de planificación, valida su idoneidad para tareas que requieren precisión matemática y estructura rigurosa, como la determinación de métricas globales desde una perspectiva puramente modular, y gracias al uso de clases y abstracciones bien definidas, se pueden lograr modelar dichos comportamientos para resolver problemas complejos de la cotidianidad informática de forma más efectiva.

6. Enlaces de Entrega.

Finalmente, en esta sección se disponen de los enlaces correspondientes a cada elemento a evaluar.

Primero, el enlace al repositorio público de GitHub donde reposa el código fuente y los archivos de pruebas utilizados en este desarrollo:

<https://github.com/davidtaborda2025/Desarrollo-Primer-Parcial---Sistemas-Operativos.git>

Paralelamente, aquí también se añade el enlace al video de la sustentación del desarrollo aquí explicado, ubicado en YouTube:

https://youtu.be/Wj6pWREPMHg?si=od_1UoEe57Cv0zVG